From Red Flags to Detection Rules: An LLM-driven Pipeline for Real-Time GOOSE Intrusion Detection and Prevention

Lucas A. Martins[1]*, Silvio E. Quincozes[1,2],

[1] [1*]Universidade Federal de Uberlândia (UFU) – Uberlândia, Brasil
[2] Universidade Federal do Pampa (UNIPAMPA) – Alegrete, Brasil

{⋆lucas.martins, sequincozes}@ufu.br

Abstract. *Specification-based Intrusion Detection Systems (IDSs) are widely used in IEC 61850 substations, but rely on manually crafted rules derived from expert knowledge. This paper presents an LLM-driven pipeline that automates the generation of detection rules for real-time GOOSE intrusion detection and prevention. The approach uses labeled communication samples to identify red flags and transform them into intrusion-detection rules. The pipeline supports plug-and-play execution, reproducibility, and parameterized control. A proof-of-concept using the ERENO dataset shows that the generated rules effectively detect anomalous behavior with low operational overhead, indicating that LLM-driven automation is a viable approach for specification-based IDS.*

1. Introduction

Digital substations based on the IEC–61850 standard face several cybersecurity challenges and are increasingly exposed to evolving cyberattacks, including Denial-of-Service (DoS), Message Injection, and Masquerade attacks [Malik et al. 2022]. In this context, Intrusion Detection Systems (IDSs) play a fundamental role in ensuring the reliability, safety, and operational continuity of such critical infrastructures [Quincozes et al. 2021].

Among the existing detection approaches, specification-based IDSs are widely adopted in IEC–61850 environments due to their low computational overhead and interpretability. These systems rely on expert-defined rules that describe the expected behavior of network communications, flagging deviations as potential intrusions [Quincozes et al. 2021]. Several works have proposed rule sets targeting the Generic Object Oriented Substation Event (GOOSE)-based communication, covering aspects such as protocol integrity, temporal consistency, and network behavior [Hong and Liu 2019, Hong et al. 2014a, Hong et al. 2014b, Yang et al. 2016a, Yang et al. 2016b, Kwon et al. 2015].

Despite their relevance, specification-based IDSs present a fundamental limitation: the dependence on manually crafted rules derived from domain expertise. This process is inherently time-consuming, difficult to scale, and hard to adapt to new attack patterns or evolving system behaviors. Moreover, prior works have primarily focused on proposing and qualitatively analyzing such rules, with limited efforts toward systematic, data-driven rule generation or automated validation pipelines.

Recently, the availability of realistic intrusion detection datasets, such as ERENO–IEC–61850 [Quincozes et al. 2022], has enabled more structured experimentation by providing labeled GOOSE communication traces containing both normal behavior and

multiple attack classes. These datasets create an opportunity to bridge the gap between data-driven analysis and specification-based IDS design, enabling the extraction of behavioral patterns directly from observed communication samples.

In parallel, Large Language Models (LLMs) have emerged as powerful tools for pattern extraction, reasoning, and structured knowledge generation from complex data representations. While widely explored in natural language processing and software engineering tasks, their potential for automating the generation of specification-based detection rules from network data remains largely unexplored.

This work presents an LLM-driven pipeline that automates the generation of detection rules for real-time GOOSE intrusion detection and prevention. The approach consumes labeled communication samples to derive *red flags*, which are transformed into executable intrusion-detection rules and deployed in a simulated programmable switching environment using realistic traffic generated by the ERENO framework. The pipeline supports plug-and-play execution, reproducibility, and parameterized control, reducing reliance on expert-defined rules while improving adaptability to evolving attack scenarios.

2. Specification-based IDSs for GOOSE Communication in Substations

The IEC–61850-8-1 standard specifies communication networks and systems for digital substations, including the GOOSE protocol, which supports time-critical protection and control operations through a publisher/subscriber model over Ethernet [Commission 2003]. A typical GOOSE frame is shown in Figure 1. These messages carry operational fields such as `stNum`, `sqNum`, timestamps, and dataset references, whose consistency is essential for correct system behavior. However, since GOOSE was not originally designed with strong native security mechanisms, it is vulnerable to cyberattacks such as

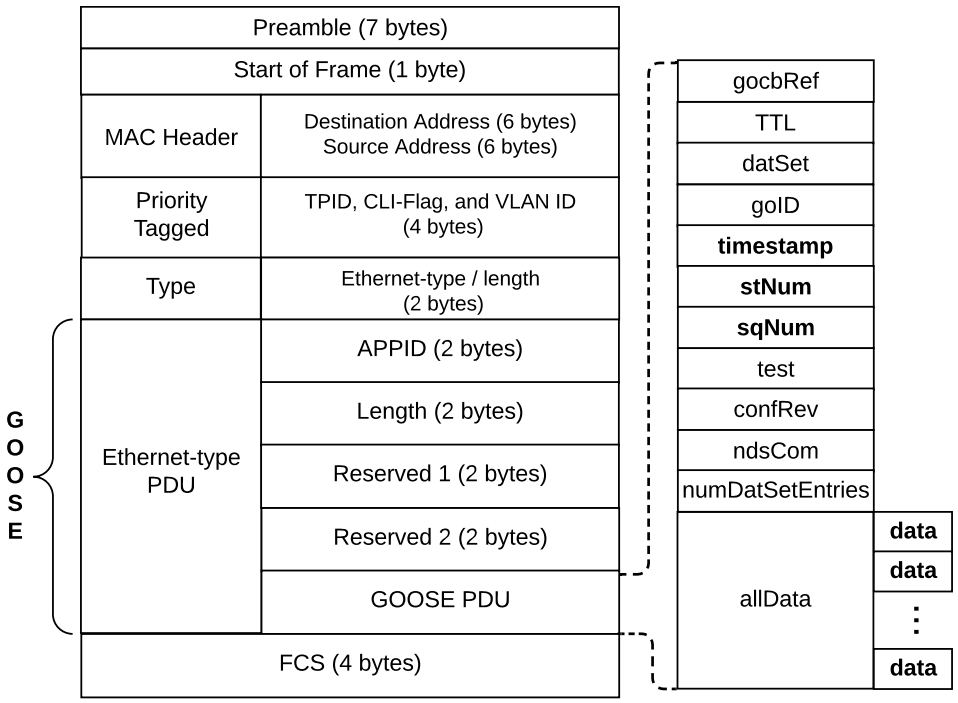

Figura 1. Ethernet frame of GOOSE messages.

injection, replay, and denial-of-service, which may compromise the coordination of protection functions [Malik et al. 2022].

In this context, specification-based detection is particularly suitable because many GOOSE fields naturally induce behavioral invariants that can be encoded as rules. Prior works explored this idea by checking structural fields such as destination MAC, `TPID`, `ethType`, `gooseAppid`, and `timeAllowedToLive` [Hong and Liu 2019, Hong et al. 2014a, Hong et al. 2014b, Yang et al. 2016a, Yang et al. 2016b], as well as consistency-related fields such as `gocbRef`, `datSet`, `goID`, `t`, `StNum`, and `SqNum` [Hong and Liu 2019, Hong et al. 2014a, Hong et al. 2014b, Yang et al. 2016b].

The literature also includes rules over communication dynamics, such as message frequency, recency, bytes per second, and packets per second [Kwon et al. 2015, Yang et al. 2016b]. Together, these works show that protocol fields, inter-message relationships, and traffic-level metrics can all serve as the basis for GOOSE intrusion-detection rules.

Because such rules are typically lightweight and directly evaluated over message attributes and simple traffic statistics, they are particularly attractive for real-time environments such as digital substations, where detection latency and operational overhead must remain low. The main challenge, however, is that these rules still depend on expert knowledge to be manually specified and tailored to specific attack types [Quincozes et al. 2021]. To support controlled experimentation, the ERENO–IEC–61850 dataset provides labeled samples of normal and malicious GOOSE traffic generated under realistic substation conditions [Quincozes et al. 2022]. This enables the systematic analysis of communication patterns and offers a practical basis for automatically deriving detection logic from observed data rather than specifying rules entirely by hand.

3. Proposed Architecture

In contrast to traditional specification-based IDSs, in which detection rules are manually defined by domain experts, our proposal delivers an automated pipeline that derives detection logic directly from labeled GOOSE communication samples. The architecture is designed to transform observed attack/normal patterns into executable rules and evaluate them in a simulated real-time environment.

Figure 2 illustrates the overall workflow of the proposed architecture. From left to right, the pipeline starts with a labeled GOOSE dataset, which serves as the sole input source for the generation process. These samples are then processed by an LLM-driven *Red Flag Extraction* stage, responsible for identifying suspicious patterns, behavioral inconsistencies, and relevant discriminative cues associated with malicious or benign traffic. Next, the extracted red flags are forwarded to a *Rule Generation* stage, where they are converted into executable Python detection rules. Finally, the generated rules are deployed into a *Switch Simulation* stage, which emulates a programmable switching environment that processes GOOSE messages and applies the generated logic to perform intrusion detection and prevention decisions in real time.

The architecture is organized into four main stages: *source ingestion*, *red flag extraction*, *rule generation*, and *simulated deployment*. Together, these stages form an end-to-end pipeline that bridges data-driven analysis and specification-based detection.

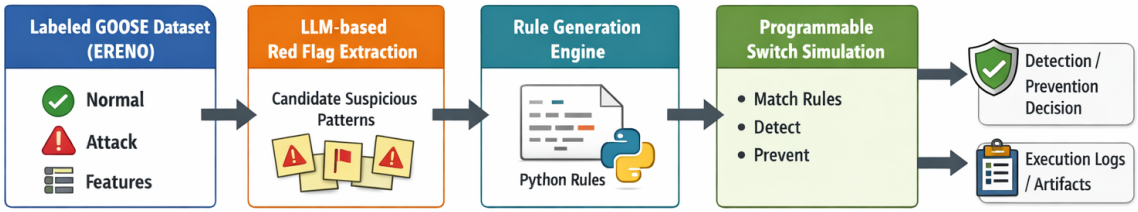

Figura 2. Overview of the proposed architecture.

3.1. Source Ingestion

The pipeline begins with labeled GOOSE communication samples containing both normal and attack instances. In our proof of concept, these samples are provided by the ERENO framework, which generates realistic substation traffic under both normal operating conditions and cyberattack scenarios. Each sample is represented by protocol-level, temporal, and derived features, allowing the pipeline to observe both individual message properties and inter-message behavior.

At this stage, the source ingestion component is responsible for loading the dataset, organizing the available features, and preparing structured prompts for the LLM. Rather than requiring manually written specifications, the system uses the labeled samples themselves as the knowledge source from which suspicious patterns will be inferred.

3.2. Red Flag Extraction

Once the samples are loaded, the next stage consists of extracting red flags from the data. A red flag represents a potentially relevant condition, inconsistency, or behavioral deviation that may help distinguish malicious from benign GOOSE traffic. Examples include abnormal transitions in `StNum` or `SqNum`, inconsistent timing-related behavior, suspicious combinations of protocol fields, or abrupt variations in derived features such as `stDiff`, `sqDiff`, and `timestampDiff`.

The LLM receives structured descriptions of the labeled samples and produces candidate red flags in a semi-structured form. In practice, this stage acts as an abstraction layer between raw data and executable detection logic: instead of directly generating code from samples, the pipeline first elicits higher-level suspicious conditions that can later be formalized as rules. This intermediate representation improves traceability and makes the generation process easier to inspect.

3.3. Rule Generation

The extracted red flags are then passed to the rule generation stage, which translates them into executable Python rules. Each rule is expressed as a conditional statement over one or more input features and produces a detection outcome when its condition is satisfied. This stage is responsible for converting descriptive patterns into operational logic that can be evaluated online.

The generated rules follow the same spirit as traditional specification-based IDS rules, since they operate over protocol fields, inter-message consistency, and traffic-level indicators. However, unlike prior approaches, the rules are not written manually by specialists. Instead, they are automatically derived from labeled observations, reducing the

dependence on handcrafted expert knowledge and allowing the detection logic to better reflect the patterns present in the analyzed data.

3.4. Simulated Deployment

After generation, the rules are automatically deployed into a simulated programmable switch. This component emulates a real-time packet-processing environment in which incoming GOOSE samples are sequentially inspected and checked against the generated rules. For each processed sample, the simulator applies the rule set, determines whether the traffic should be considered benign or suspicious, and records the corresponding decision.

The purpose of this stage is not to reproduce the full behavior of a hardware switch, but to provide a controlled environment for evaluating the feasibility of rule execution under real-time constraints. Because the generated rules are implemented as lightweight Python conditions over packet attributes and simple derived metrics, they preserve the low-overhead nature expected from specification-based detection and are suitable for time-sensitive environments such as digital substations.

3.5. Outputs and Artifacts

Besides the generated rules themselves, the architecture produces a set of artifacts that support inspection and reproducibility. These include the extracted red flags, the Python rule files, execution logs from the simulated switch, and detection outputs for the processed samples. Such artifacts make it possible to analyze not only whether the pipeline detects malicious behavior, but also which inferred conditions were converted into rules and how they behaved during deployment.

Overall, the proposed architecture operationalizes a complete path from labeled GOOSE samples to executable detection logic. By separating the workflow into explicit stages, it becomes possible to inspect, reproduce, and evolve each part of the pipeline while preserving the practical benefits of specification-based detection.

4. Tool Demonstration: Parameters, Execution and Outputs

EXEMPLO DE OUTRO TRABALHO

This section describes how to obtain and execute the *CounselorNode* tool in a reproducible manner, from repository cloning to a minimal runtime execution. The tool is publicly available on GitHub and can be executed locally as a standalone counselor node.

4.1. Repository and environment setup

The first step is to clone the official repository:

```
git clone https://github.com/sequincozes/CounselorNode.git
cd CounselorNode
```

To avoid dependency conflicts and ensure reproducibility, we recommend running the tool inside a virtual environment. For example, on Linux or macOS:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

On Windows, the activation step changes accordingly:

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
```

Once the repository is cloned and dependencies are installed, the tool can be executed through its entry point module. A minimal execution (single node) follows:

```
python -m counselornode.run_node 5000
```

At startup, the node automatically detects its local IP address, initializes the networking layer, and builds the machine learning engine. After a short warm-up period, the node enters a continuous loop where it iterates over dataset samples, classifies them, and applies the protocol logic to decide whether the sample is trusted or if a conflict is detected. The execution can be terminated at any time using `Ctrl+C`.

To access detailed information on how to reproduce our PoC, please read the *README.md* content at `https://github.com/sequincozes/CounselorNode`.

5. Proof-of-Concept (PoC)

5.1. Detection Capability

5.2. Real-time Feasibility

6. Final Remarks

Referências

Commission, I. E. (2003). *Communication networks and systems in substations - Part 8-1: Specific communication service mapping (SCSM) - Mappings to MMS (ISO 9506-1 and ISO 9506-2) and to ISO/IEC 8802-3*. IET.

Hong, J. and Liu, C. (2019). Intelligent electronic devices with collaborative intrusion detection systems. *IEEE Transactions on Smart Grid*, 10(1):271–281.

Hong, J., Liu, C., and Govindarasu, M. (2014a). Detection of Cyber Intrusions Using Network-Based Multicast Messages for Substation Automation. In *Innovative Smart Grid Technologies (ISGT)*, pages 1–5. IEEE.

Hong, J., Liu, C.-C., and Govindarasu, M. (2014b). Integrated anomaly detection for cyber security of the substations. *IEEE Transactions on Smart Grid*, 5(4):1643–1653.

Kwon, Y., Kim, H. K., Lim, Y. H., and Lim, J. I. (2015). A behavior-based intrusion detection technique for smart grid infrastructure. In *2015 IEEE Eindhoven PowerTech*, pages 1–6. IEEE.

Malik, H., Alotaibi, M. A., and Almutairi, A. (2022). Cyberattacks identification in iec 61850 based substation using proximal support vector machine. *Journal of Intelligent & Fuzzy Systems*, 42(2):1213–1222.

Quincozes, S. E., Albuquerque, C., Passos, D., and Mossé, D. (2021). A survey on intrusion detection and prevention systems in digital substations. *Computer Networks*, 184:107679.

Quincozes, S. E., Passos, D., Albuquerque, C., Mossé, D., and Ochi, L. S. (2022). *ERENO: AN EXTENSIBLE TOOL FOR GENERATING REALISTIC IEC–61850 INTRUSION DETECTION DATASETS*. PhD thesis, Universidade Federal Fluminense.

Yang, Y., McLaughlin, K., Gao, L., Sezer, S., Yuan, Y., and Gong, Y. (2016a). Intrusion detection system for IEC 61850 based smart substations. In *2016 IEEE Power and Energy Society General Meeting (PESGM)*, pages 1–5. IEEE.

Yang, Y., Xu, H.-Q., Gao, L., Yuan, Y.-B., McLaughlin, K., and Sezer, S. (2016b). Multi-dimensional intrusion detection system for IEC 61850-based SCADA networks. *IEEE Transactions on Power Delivery*, 32(2):1068–1078.

A. Appendix: Recommended parameters

Figure **??** summarizes parameters grouped by purpose, combining (i) the command-line interface and (ii) the JSON configuration used by the `CounselorNode`. These parameters can be defined in the *peer_config.json* file at the CounselorNode repository.